

Chương này trình bày các giải pháp kỹ thuật và đóng góp nổi bật của đồ án trong quá trình xây dựng hệ thống hiển thị giao thông 3D từ dữ liệu mô phỏng SUMO. Các nội dung về kiến trúc và kết quả minh họa đã được trình bày ở chương trước (xem Hình ??); trong chương này, báo cáo tập trung vào những điểm mang tính “giải quyết vấn đề” và “đóng gói thành giải pháp” để có thể áp dụng lại cho các kịch bản khác.

0.1 Giải pháp giao tiếp thời gian thực qua TCP: tách khung thông điệp bằng marker <END>

0.1.1 Bài toán/vấn đề

Trong hệ thống, Unity cần nhận dữ liệu trạng thái theo từng bước mô phỏng (xe, đèn giao thông, người đi bộ) từ phía Python điều khiển SUMO. Một lựa chọn đơn giản là dùng TCP socket và gửi chuỗi JSON. Tuy nhiên, TCP là giao thức hướng luồng (stream-oriented), không bảo đảm ranh giới “mỗi lần gửi” tương ứng với “mỗi lần nhận”. Vì vậy, trong thực tế có thể xảy ra các tình huống:

- Một lần `recv` nhận được **một phần** thông điệp JSON (thiếu dữ liệu, JSON không parse được).
- Một lần `recv` nhận được **nhiều** thông điệp JSON gộp lại (Unity không biết tách ở đâu).

Ngoài ra, ứng dụng cần hỗ trợ các lệnh điều khiển từ Unity (Pause/Resume/End) và có thể cập nhật một số tham số chạy mô phỏng (ví dụ `timeStep`). Điều này đòi hỏi cơ chế nhận thông điệp điều khiển ổn định, tránh “kẹt” luồng nhận và đảm bảo đóng kết nối gọn.

0.1.2 Giải pháp

Đồ án sử dụng TCP socket để truyền thông điệp dạng JSON giữa Python và Unity, đồng thời bổ sung cơ chế **tách khung** (message framing) bằng marker <END>.

Nguyên tắc đóng gói. Mỗi payload JSON được gửi như một chuỗi UTF-8, sau đó gửi thêm marker <END> để biểu thị kết thúc bản tin. Khi payload lớn, hàm `send_data` trong `network.py` chia chuỗi JSON thành các đoạn (chunk) theo kích thước bộ đệm và `sendall` tuần tự; marker luôn được gửi ở cuối để phía nhận biết điểm kết thúc.

Nguyên tắc bóc tách ở phía nhận. Phía nhận không parse theo từng `recv` mà duy trì một *receive buffer* cho mỗi socket. Hàm `receive_message/receive_data` đọc thêm dữ liệu cho tới khi gặp marker <END>, sau đó tách ra đúng **một** thông điệp hoàn chỉnh và giữ lại phần dư cho lần nhận tiếp theo. Thiết kế buffer theo

socket giải quyết đúng bản chất “TCP là stream”, tránh cả hai lỗi: JSON bị cắt giữa chừng và nhiều JSON bị dính lại.

Tích hợp điều khiển mô phỏng. Trên nền giao thức này, chương trình hỗ trợ hai nhóm bản tin:

- **Bản tin dữ liệu:** trạng thái theo step gồm `vehicles`, `trafficLights`, `pedestrians`.
- **Bản tin điều khiển:** các lệnh `Pause`, `Resume`, `Simulation end` và (tùy chọn) một JSON cấu hình đơn giản để cập nhật `timeStep`.

0.1.3 Kết quả đạt được

Giải pháp tách khung bản tin giúp kênh giao tiếp Unity–Python ổn định và dễ mở rộng hơn trong quá trình mô phỏng thời gian thực. Cụ thể:

- Giảm lỗi parse JSON do phân mảnh/gộp gói trên TCP stream.
- Cho phép truyền đồng thời dữ liệu trạng thái và lệnh điều khiển theo một cơ chế nhận thống nhất.
- Hỗ trợ tạm dừng/tiếp tục và kết thúc mô phỏng an toàn từ Unity, tạo trải nghiệm vận hành ổn định khi thử nghiệm.

0.2 Giải pháp điều phối vòng lặp mô phỏng, đa cổng kết nối và dừng hệ thống an toàn

0.2.1 Bài toán/vấn đề

Hệ thống mô phỏng–hiển thị gồm nhiều tác vụ chạy song song: (i) chạy mô phỏng SUMO theo step, (ii) gửi dữ liệu trạng thái cho Unity theo thời gian thực, (iii) nhận lệnh điều khiển (`Pause/Resume/End`), và (iv) (tùy chọn) nhận cập nhật đối tượng từ Unity. Trong bối cảnh này, hai nhóm vấn đề chính cần giải quyết là:

- **Tránh treo (deadlock) do I/O chặn:** các lời gọi `accept/recv` có thể chặn vô thời hạn, khiến chương trình không phản ứng khi cần dừng.
- **Dừng hệ thống gọn:** khi kết thúc mô phỏng cần đảm bảo đóng socket, dừng thread và ghi kết quả (CSV/JSON) đầy đủ; tránh trạng thái dở dang hoặc rò rỉ tài nguyên.

0.2.2 Giải pháp

Đề án điều phối chương trình theo hướng đa luồng (multi-threading) và tách kênh kết nối theo mục đích, bám sát cách triển khai trong `main.py`:

- **Tách cổng theo kênh dữ liệu:** hệ thống khởi tạo ba server socket: port 5050 cho dữ liệu chính (`RoadData/TrafficData`), port 5053 cho kênh nhận cập nhật

xe từ Unity (nếu dùng), và port 5054 cho kênh lệnh điều khiển (shutdown/pause/resume).

- **Cơ chế dừng tập trung:** sử dụng cờ `stop_event` để báo dừng cho toàn hệ thống; khi Unity gửi `Simulation end` hoặc khi mô phỏng kết thúc tự nhiên, cờ này được set và vòng lặp chờ ở tiến trình chính thoát.
- **Tạm dừng/tiếp tục an toàn:** dùng `pause_event` kết hợp với `sleep` để tạm dừng vòng lặp `simulationStep` mà không phá trạng thái TraCI.
- **Cập nhật tham số thời gian chạy:** `time_step` được bảo vệ bởi `time_step_lock` để cập nhật từ lệnh điều khiển, tránh race condition khi thread mô phỏng đang đọc giá trị này.
- **Shutdown gọn:** khi dừng, chương trình đóng các socket server, sau đó `join` các thread nền với timeout; luồng server trong `network.py` cũng có xử lý trường hợp socket đã bị đóng (trên Windows thường gặp `winerror 10038`) để thoát vòng `accept`.

0.2.3 Kết quả đạt được

Giải pháp điều phối giúp chương trình vận hành ổn định hơn trong quá trình thử nghiệm, đặc biệt ở các thao tác thường gặp như dừng mô phỏng hoặc tạm dừng để quan sát. Việc dừng gọn cũng đảm bảo các bước hậu xử lý (đóng TraCI, ghi file CSV/JSON tổng hợp) được thực hiện đầy đủ, qua đó tăng tính lặp lại của thực nghiệm.

0.3 Giải pháp sinh mạng SUMO để benchmark theo phương pháp vét cạn từ lưới .

0.3.1 Bài toán/vấn đề

Trong bối cảnh kiểm thử và đo đặc hiệu năng (benchmark), bài toán quan trọng không chỉ là “có bản đồ để chạy” mà còn là tạo được một mạng có cấu trúc **để điều khiển và có thể mở rộng kích thước** nhằm quan sát tải hệ thống (số lượng node/edge tăng theo kích thước bản đồ). Nếu phụ thuộc vào bản đồ thật (OpenStreetMap) hoặc dựng thủ công, việc đảm bảo tính lặp lại và điều chỉnh độ phức tạp của mạng sẽ khó và tốn thời gian.

Đồ án sử dụng đầu vào dạng lưới ký tự (@ là tường, . là lối đi). Tuy nhiên, thay vì chỉ sinh node tại các “nút giao” suy ra từ hình học mê cung, dự án hiện tại chọn cách **vét cạn**: *mỗi ô . được coi là một node để tạo ra mạng dày (dense) phục vụ benchmark.*

Từ biểu diễn này, cần tạo được các tệp đầu vào chuẩn của SUMO:

- Vị trí các nút (junction) hợp lệ để tạo `.nod.xml`.

- Các cạnh có hướng (edge) kết nối các nút để tạo `.edg.xml`.
- Tập kết nối/crossing (`.con.xml`) để hỗ trợ người đi bộ và các điểm qua đường.
- Quy tắc làn (lane) cho xe và người đi bộ để phục vụ hiển thị đa đối tượng.

0.3.2 Giải pháp

Trong dự án hiện tại, giải pháp sinh bản đồ benchmark được hiện thực trong `naive_map_creator.py` và dùng các hàm ghi XML trong thư mục `SUMO_xml`. Quy trình chính:

(1) Đọc lưới và vết cạnh node. Hàm `parse_map_file` đọc phần header (width, height) và ma trận grid. Sau đó `get_node_pos_map` duyệt toàn bộ lưới; với mỗi ô `.`, tạo một node `n_i` và lưu vào `pos_map` theo khoá tọa độ làm tròn 2 chữ số (ổn định khi so khớp).

(2) Sinh cạnh theo 4 hướng. Để tạo `.edg.xml`, hàm `write_maze_edges_to_xml` (trong `SUMO_xml/create_map_from_maze.py`) duyệt từng node và quét theo 4 hướng (phải, xuống, trái, lên) cho tới khi gặp tường `@`. Node đầu tiên gặp được theo hướng đó sẽ được nối bằng một cạnh có hướng. Khi mọi ô `.` đều là node, phép quét này tương đương với việc nối các ô lối đi kề nhau, tạo ra một mạng dày để benchmark.

(3) Ghi file node/crossing và chuẩn hóa làn. `write_nodes_to_xml` ghi `.nod.xml` và sử dụng hệ số tỉ lệ (ví dụ `scale=40.0`) để chuyển tọa độ lưới sang đơn vị mét trong SUMO. Đồng thời `write_crossings_to_con_xml` sinh `.con.xml` để hỗ trợ người đi bộ. Khi ghi edge, cấu trúc làn được chuẩn hóa theo hướng có làn đi bộ (`allow="pedestrian"`) và các làn xe (`disallow="pedestrian"`) nhằm phục vụ hiển thị đa đối tượng.

Ghi chú về các phương án không vết cạnh. Trong thư mục `SUMO_xml` còn có các thuật toán sinh bản đồ khác như `create_map_from_maze.py` (suy luận node từ hình học tường/độ bậc để tạo mạng thưa hơn) và `create_city_map.py` (sinh bản đồ theo kiểu “thành phố” với các heuristic kiểm tra kết nối). Các phương án này có mã nguồn và có thể dùng cho kịch bản thực tế hơn, nhưng **không được sử dụng trong pipeline benchmark hiện tại** (xem cách gọi trong `main.py`).

0.3.3 Kết quả đạt được

Giải pháp vết cạnh giúp sinh nhanh các mạng benchmark có độ phức tạp điều chỉnh được theo kích thước lưới, nhờ đó:

- Tạo mạng dày và có tính lặp lại cao để đo tải hệ thống (số lượng node/edge

tăng theo kích thước bản đồ).

- Duy trì cú pháp dữ liệu SUMO chuẩn (node/edge/connection) để tiếp tục dùng `netconvert` và các công cụ chuẩn.
- Bảo đảm có làn người đi bộ và cấu trúc dữ liệu phù hợp để hệ thống hiển thị 3D biểu diễn đa đối tượng.

0.4 Giải pháp sinh tuyến đường (route) và lọc cặp điểm không liên thông

0.4.1 Bài toán/vấn đề

Sau khi có mạng đường, để tạo mô phỏng cần sinh tệp tuyến đường `.rou.xml`. Nếu chọn cặp điểm đầu–cuối (OD pair) ngẫu nhiên mà không kiểm tra tính liên thông, SUMO có thể không tìm thấy đường đi, dẫn tới lỗi hoặc làm giảm chất lượng kịch bản thử nghiệm. Bên cạnh đó, cần tạo kịch bản đa dạng (đi ngang mạng, đi từ trong ra ngoài, v.v.) để quan sát các tình huống giao thông khác nhau.

0.4.2 Giải pháp

Đồ án xây dựng bộ sinh tuyến trong `SUMO_xml/route_gen.py` theo hai ý chính:

- **Chiến lược chọn OD pair:** các hàm `cross_side_gen`, `swap_side_gen`, `in_out_gen`, `out_in_gen` chia tập junction theo thống kê tọa độ (theo trục x hoặc theo bán kính so với tâm) để tạo các cặp OD có đặc trưng mong muốn.
- **Lọc tính khả thi:** trước khi ghi flow, hàm `filter_feasible_pairs` xây dựng đồ thị kề từ file edge và dùng BFS để kiểm tra từ điểm xuất phát có thể tới điểm đích hay không. Những cặp không khả thi được loại bỏ.

Với người đi bộ, hệ thống có thể sinh flow riêng và gán tham số `impatience` để thay đổi hành vi chờ đợi tại crossing.

Trong quá trình chạy mô phỏng, đồ án sử dụng tùy chọn `-junction-taz` khi khởi chạy SUMO để cho phép coi ID của junction là điểm đầu/điểm cuối hợp lệ của chuyến đi. Thiết lập này phù hợp với cách sinh tuyến dựa trên cặp nút giao trong file route.

0.4.3 Kết quả đạt được

Giải pháp giúp giảm đáng kể tình huống “không tìm được đường đi” khi sinh tuyến tự động, đồng thời tạo được nhiều kiểu kịch bản (đi chéo mạng, đi xuyên tâm, v.v.) phục vụ kiểm thử hiển thị. Việc lọc khả thi cũng làm cho quá trình thử nghiệm có tính lặp lại cao hơn vì tránh phụ thuộc vào may rủi của chọn ngẫu nhiên.

0.5 Giải pháp chuẩn hóa dữ liệu TraCI cho hiển thị 3D (xe, đèn giao thông, người đi bộ)

0.5.1 Bài toán/vấn đề

TraCI cung cấp trạng thái mô phỏng theo thời gian (vị trí, tốc độ, góc hướng, trạng thái đèn, ...). Tuy nhiên, dữ liệu thô từ SUMO cần được chuẩn hóa để Unity có thể sử dụng nhất quán trong cập nhật 3D. Các vấn đề chính gồm:

- Dữ liệu của các loại đối tượng khác nhau cần được đưa về một “hợp đồng dữ liệu” thống nhất (ID, type, position, direction/forward, state).
- Với xe, cần suy ra vector hướng từ góc phương vị để xoay mô hình 3D đúng chiều.
- Với đèn giao thông, cần xác định vị trí đại diện của tín hiệu dựa trên hình học làn đường (lane shape).

0.5.2 Giải pháp

Đồ án xây dựng các hàm đọc dữ liệu trong thư mục HelloWorld/Traffic và đóng gói lại thành JSON theo cấu trúc rõ ràng:

- **Xe:** `read_vehicles` đọc vị trí (x, y) , tốc độ, lane ID và trạng thái xi-nhan/phanh. Góc hướng θ (độ) được chuyển sang vector đơn vị $\vec{f} = (\cos \theta, \sin \theta)$ (sau khi đổi về radian) để Unity có thể xoay xe.
- **Đèn giao thông:** `read_traffic_lights` đọc chuỗi trạng thái RYG của từng traffic light và danh sách lane được điều khiển. Với mỗi lane, lấy shape và chọn điểm cuối làm vị trí đặt đèn (ở mức trừu tượng). Trạng thái tín hiệu được chuyển thành 3 biến boolean tương ứng `isRed`, `isYellow`, `isGreen` (trong đó g và G đều được coi là xanh) để Unity hiển thị.
- **Người đi bộ:** `read_pedestrians` (tương tự) đọc danh sách ID và vị trí, phục vụ hiển thị.

Tại vòng lặp mô phỏng, dữ liệu được nhóm thành một payload gồm `trafficLights`, `vehicles`, `pedestrians` và gửi sang Unity theo cơ chế ở Mục 0.1.

Về ánh xạ hệ tọa độ, SUMO biểu diễn vị trí trên mặt phẳng (x, y) . Khi đưa sang Unity 3D, dữ liệu thường được chuyển thành (x, y_{unity}, z) , trong đó z lấy từ y của SUMO và y_{unity} là độ cao (ví dụ một hằng số để đặt đối tượng lên mặt đường). Cách đặt này cũng phù hợp với cách xuất dữ liệu đèn giao thông trong `read_traffic_lights` khi sử dụng `position=[x, 5, y]`.

0.5.3 Kết quả đạt được

Giải pháp giúp Unity nhận dữ liệu có cấu trúc thống nhất và đủ thông tin để:

- Tạo/cập nhật đối tượng theo ID (vòng đời: xuất hiện–cập nhật–biến mất).
- Cập nhật chuyển động phương tiện theo vị trí và hướng.
- Hiển thị trạng thái đèn giao thông theo thời gian.

Đây là bước trung gian quan trọng để biến dữ liệu mô phỏng (mang tính kỹ thuật) thành dữ liệu hiển thị (mang tính trực quan) mà không phụ thuộc chặt vào cấu trúc nội bộ của SUMO.

0.6 Giải pháp ghi log và tổng hợp kết quả tự động theo lần chạy

0.6.1 Bài toán/vấn đề

Để đánh giá kịch bản mô phỏng và so sánh các cấu hình, hệ thống cần cơ chế ghi nhận kết quả theo từng lần chạy. Nếu chỉ quan sát trực quan, việc kết luận dễ mang tính cảm nhận và khó lặp lại. Mặt khác, dữ liệu đầu ra của SUMO có thể rất lớn; đề án cần một cách ghi log gọn, tập trung vào chỉ số cơ bản phù hợp với phạm vi thử nghiệm.

0.6.2 Giải pháp

Đề án xây dựng lớp `VehicleTripCsvLogger` trong `result.py` để ghi thông tin hành trình của xe theo đơn vị step mô phỏng. Ý tưởng:

- Ở mỗi step, lấy danh sách xe khởi hành và xe đến đích từ `TraCI`.
- Lưu lại `depart_step` theo `vehicle_id`; khi xe đến đích, tính `travel_steps` và ghi ra CSV.
- Kết thúc mô phỏng, tự động ghi một dòng tổng hợp (trung bình số step) và xuất thêm một file JSON tóm tắt (số xe, số người đi bộ quan sát được, `avg_vehicle_travel_steps`, thông tin map và tham số `pedestrian_impatience` nếu có).

Tên file kết quả được gắn timestamp để tránh ghi đè và thuận tiện truy vết theo lần chạy.

Để hạn chế mất dữ liệu khi dừng đột ngột, logger thực hiện `flush` sau khi ghi mỗi step và ghi thêm một dòng tổng hợp ở cuối file (`avg`). Đồng thời, file JSON tổng hợp được ghi riêng để Unity hoặc các công cụ phân tích có thể đọc nhanh mà không cần duyệt toàn bộ CSV.

0.6.3 Kết quả đạt được

Giải pháp tạo ra bộ dữ liệu đầu ra nhẹ, dễ dùng cho phân tích sau mô phỏng:

- CSV phục vụ kiểm tra theo từng xe (`depart/arrive/travel`).
- JSON phục vụ tổng hợp nhanh hoặc dùng làm đầu vào cho các công cụ thống

kê.

Nhờ đó, hệ thống không chỉ “hiển thị được” mà còn hỗ trợ đánh giá định lượng ở mức cơ bản, tạo nền tảng cho các hướng phát triển về sau (ví dụ phân tích theo tuyến, theo thời gian, hoặc theo khu vực).

0.7 Kết chương

Chương này đã trình bày các giải pháp và đóng góp nổi bật mang tính kỹ thuật của đồ án, tập trung vào các điểm then chốt để hiện thực hệ thống mô phỏng–hiển thị: (i) giao tiếp TCP đáng tin cậy bằng cơ chế tách khung bản tin, (ii) sinh mạng SUMO từ mê cung và chuẩn hóa làn cho đa đối tượng, (iii) sinh tuyến và lọc cặp OD không khả thi, (iv) chuẩn hóa dữ liệu TraCI cho Unity, và (v) ghi log/tổng hợp kết quả theo lần chạy. Các giải pháp này giúp hệ thống vận hành ổn định hơn và giảm đáng kể công sức chuẩn bị dữ liệu, đồng thời là cơ sở để mở rộng thêm chức năng trong phần hướng phát triển.